

Control Hijacking: Attacks

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Computer Application

- An application software is a computer software that performs a group of **pre-defined** functions or tasks
- An application provides services
 - Locally, e.g., video player
 - Remotely, e.g., web server
- The behavior of an application is determined by
 - The data/input being processed
 - The code being executed
 - The environment in which it runs

Attackers' Goals

- Compromising the security properties of a program
 - Confidentiality
 - Integrity
 - Availability
- How?
 - Take over the control of the program
 - Inject new code or new command
 - Cause it to crash

Command Injection

- Applications invoke external commands to perform specific tasks
- `system(<string>)` executes a command specified in a string by calling
 - `/bin/sh -c <string>`
- `popen()` opens a process by creating a pipe, forking, and invoking the shell as in `system()`
- If the user can control the `<string>` passed to these functions, he/she can inject arbitrary command

Command Injection - Example

```
int main(int argc, char *argv[]) {  
    char cmd[1024];  
  
    sprintf(cmd, 1024, "cat /var/log/%s", argv[1]);  
    cmd[1023] = '\0';  
  
    return system(cmd);  
}
```

```
% prog "foo; cat /etc/shadow"  
/var/log/foo: file not found  
root:$1$1Ltsefwetewt3891f135:16661:0:99999:7:::  
daemon*:16662:0:99999:7:::
```

Lessons Learned

- Invoking commands with `system()` and `popen()` is dangerous
- Input from users should always be sanitized

Application Behavior

- Control Flow
 - The order in which code is executed or interpreted
 - May represent the functionality/behavior of a program to some extent
- Control Flow Graph
 - A graphical representation of control flow of a program (or a sub-procedure)
 - All possible paths of the program that **might** be traversed during the execution of the program
 - A directed graph

Control Flow Graph

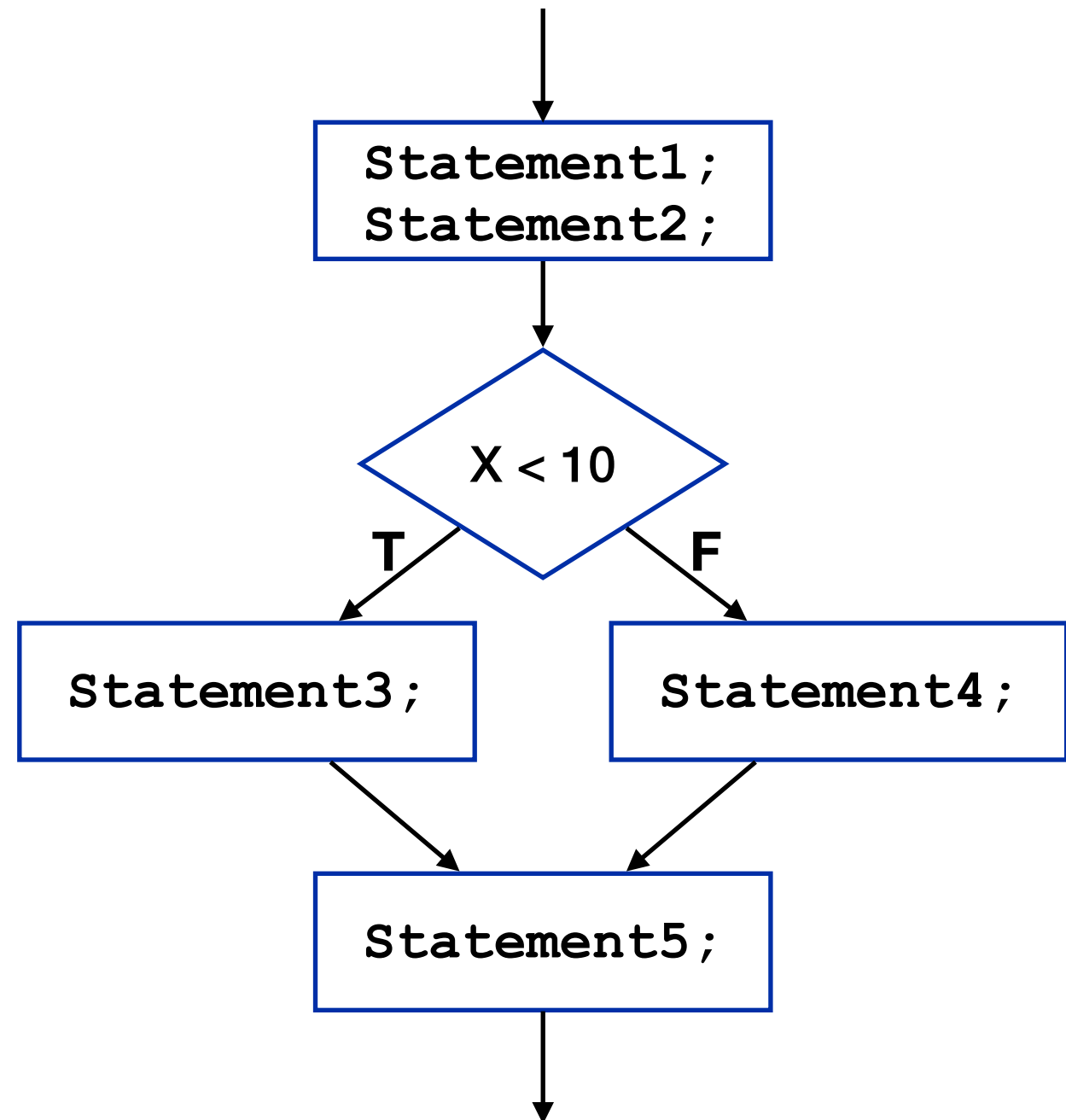
- Components
 - **Node**: a block of code without branches (**basic block**)
 - **Directed Edge**: a branch
 - A CFG shall have an entry edge and an exit edge
- Path
 - A collection of nodes connected by edges
 - A possible execution order of code
 - **Not** all paths are feasible
 - Why?

Control Flow Graph Example

Statement1;
Statement2;

if $X < 10$ then
 Statement3;
else then
 Statement4;

Statement5;



Control Flow Graph Example (Cont.)

- Class practice
 - Draw the CFG of the following code

```
Statement1;
```

```
while X < 10 then  
    Statement2;
```

```
Statement3;
```

Control Hijacking Attacks

- A class of attacks that hijack the control flow of an application
- How?
 - The application code is loaded in **memory**
- **Corrupting** the memory
 - Buffer overflow
 - Integer overflow
 - Format string exploitation
 - ...

Program State

- A program state can be described by values of
 - Processor registers
 - Its memory, e.g., stack, heap, etc.
- Processor registers
 - Several (very) quickly accessible locations
 - Load data for actual (arithmetic) operations
 - Data is copied from and back to main memory

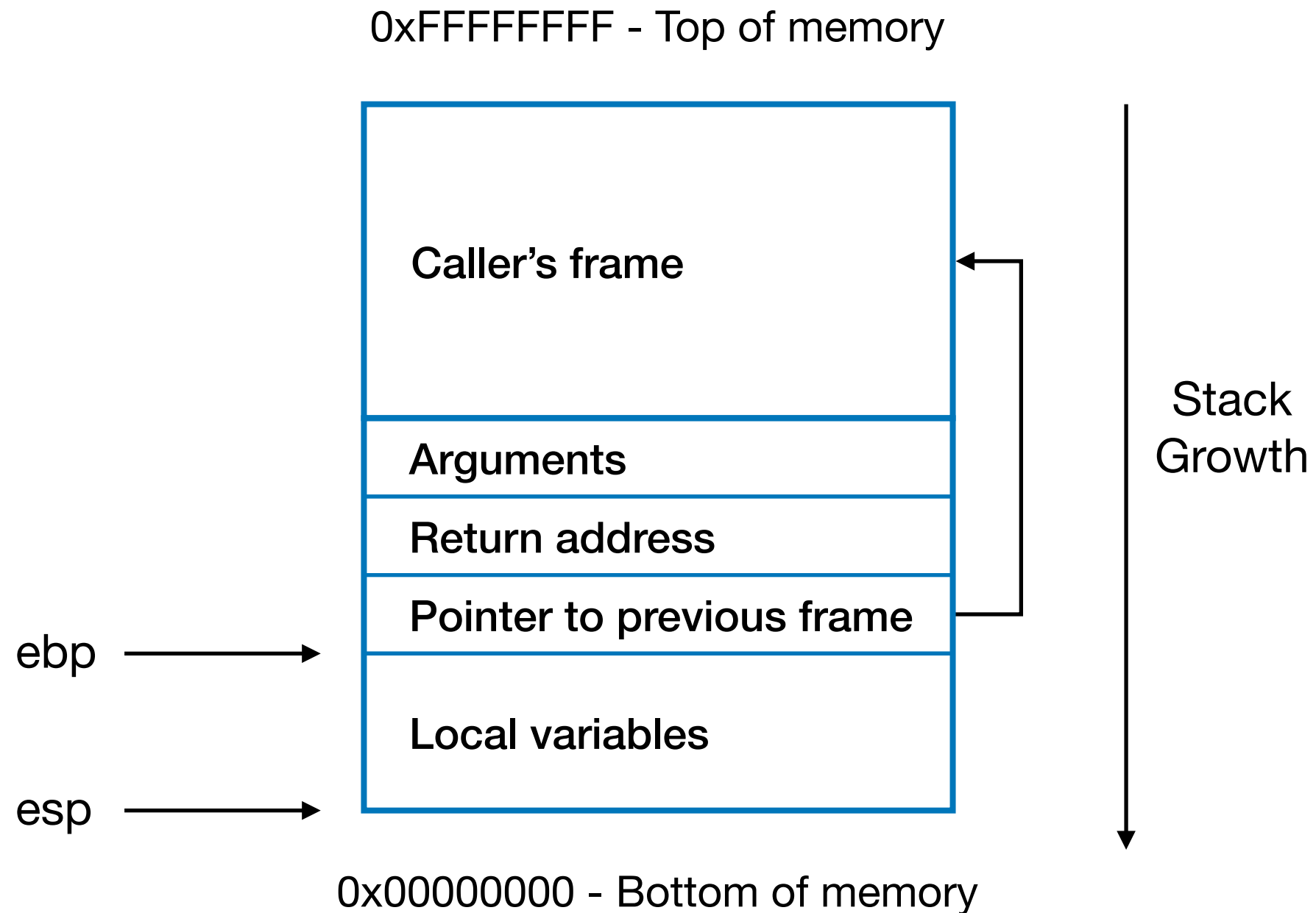
The Stack

- The stack usually grows down (towards lower memory addresses)
 - Intel, Motorola, SPARC, MIPS processors
- The stack pointer **ESP** points to the top of the stack
- **push**
 - Decrements ESP
 - Stores the value in the register at the address pointed by ESP
- **pop**
 - Stores in the register the value at the top of the stack
 - Increments ESP

Stack Frame

- The stack is composed of frames
- Frames are pushed/popped on the stack when a function is invoked/returned
- Each frame contains
 - The function's actual parameters
 - The return address
 - The pointer to the previous frame
 - The function's local variables
- The stack base pointer **EBP** stores the address of the current frame

Stack Frame Layout



Walking through Function Calls

- See separate slides from Vern Paxson

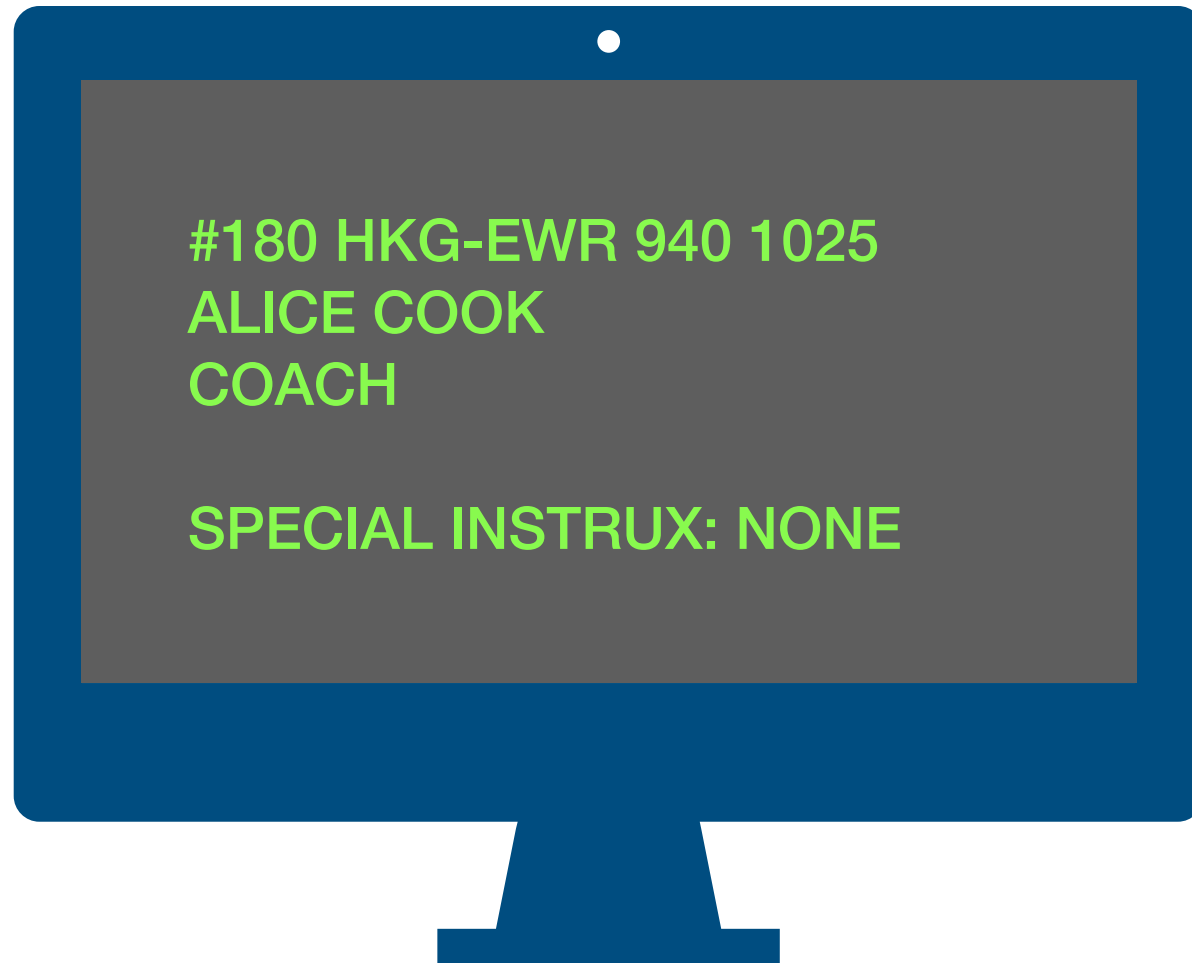
Overflows

- The **lack of boundary checking** is one of the most common mistakes in C/C++ applications
- Overflows are one of the most popular class of attacks
 - Architecture/OS dependent
 - Exploitable both locally and remotely
 - Can modify both the data and the control flow of an application
- Recent techniques have made the process of exploiting overflows **easier** if not completely automatic
- Much research has been devoted into finding vulnerabilities, developing prevention and detection mechanisms
 - Some of these mechanisms have been adopted by mainstream operating systems

Overflows - Example



Overflows - Example (Cont.)



Adult (18-64)

Alice

Cook

Date of birth*

01 - January

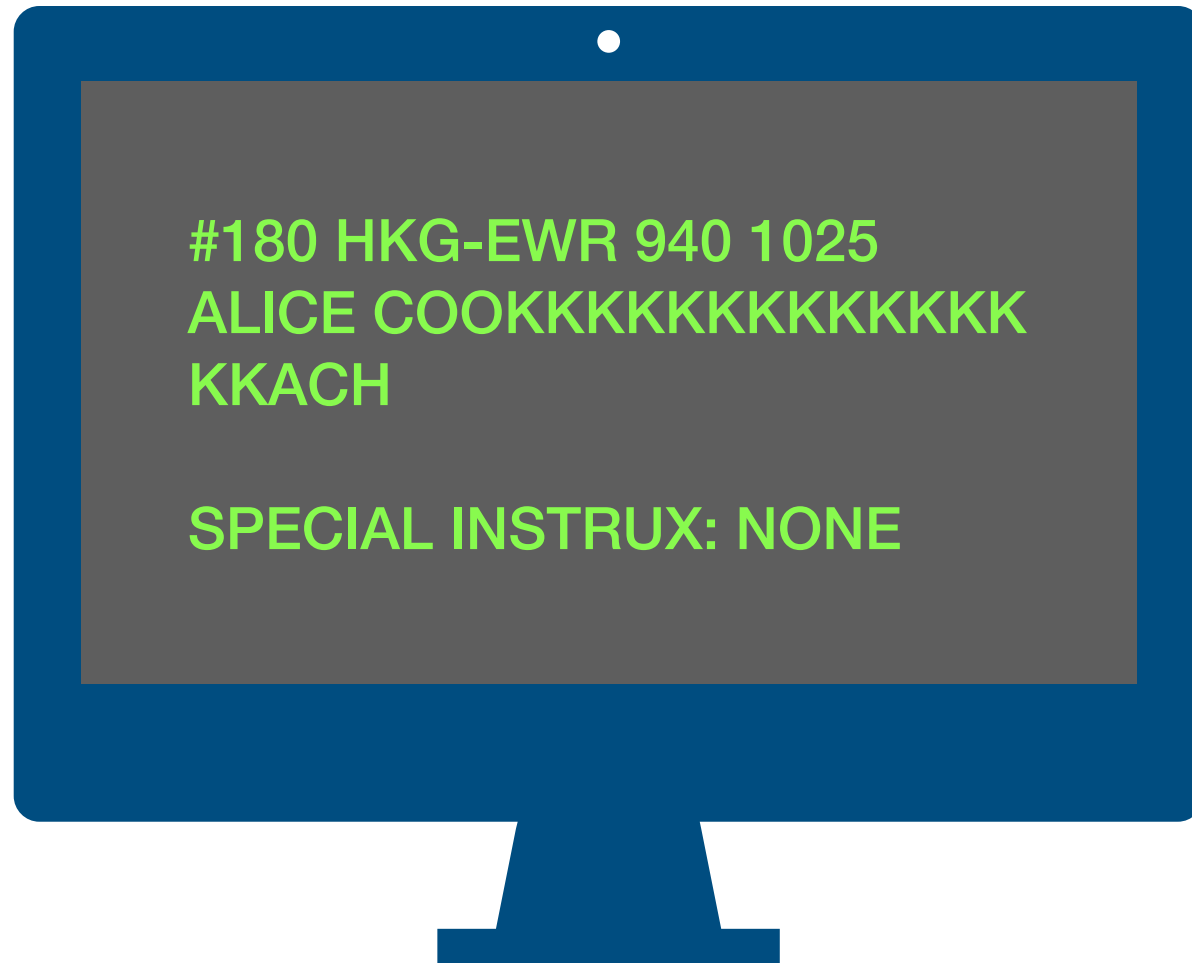


01

2000

► Known Traveler Number and Redress Number

Overflows - Example (Cont.)



Adult (18-64)

Alice

Cookkkkkkkkkkkkkkkkk

Date of birth*

01 - January

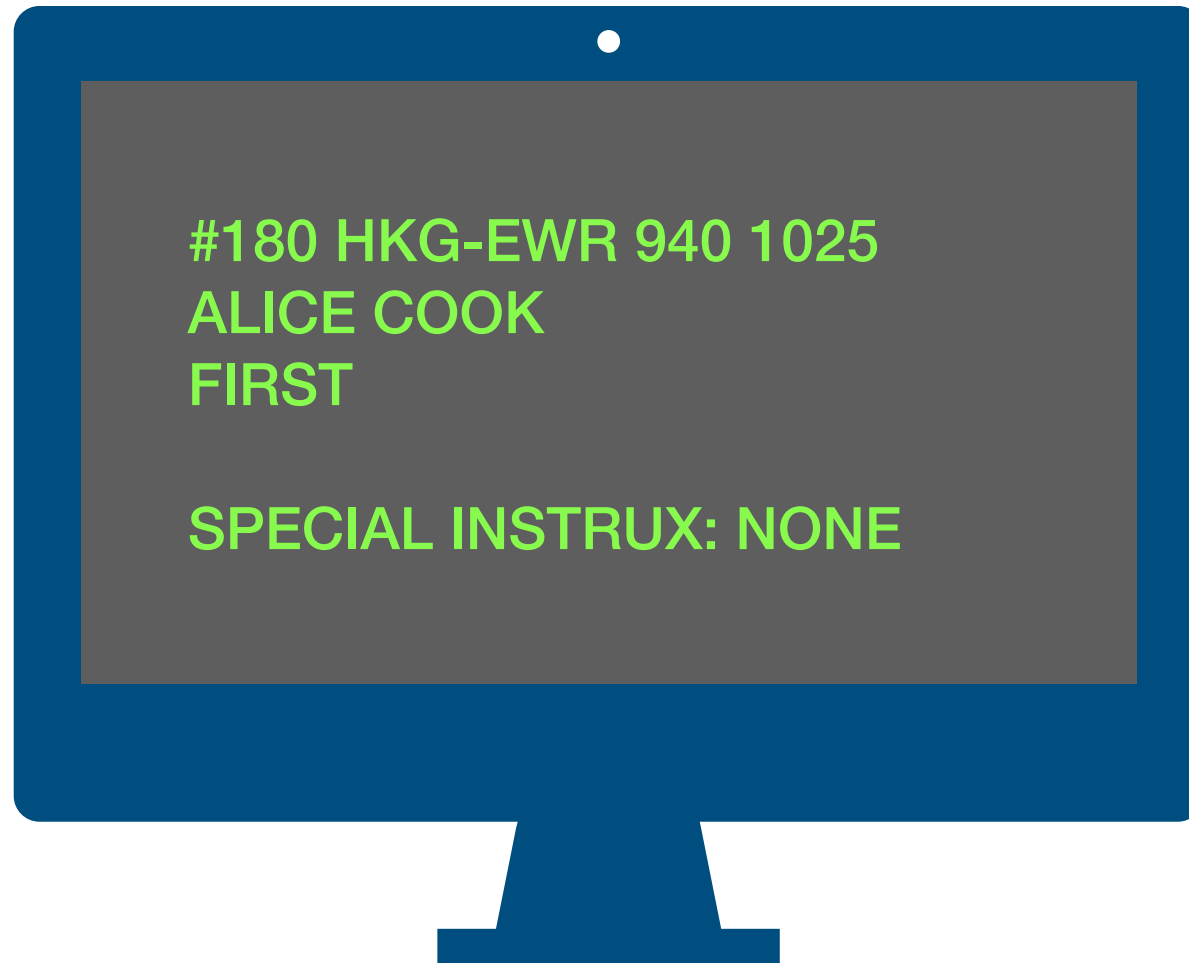


01

2000

► Known Traveler Number and Redress Number

Overflows - Example (Cont.)



Adult (18-64)

Alice

Cook First

Date of birth*

01 - January

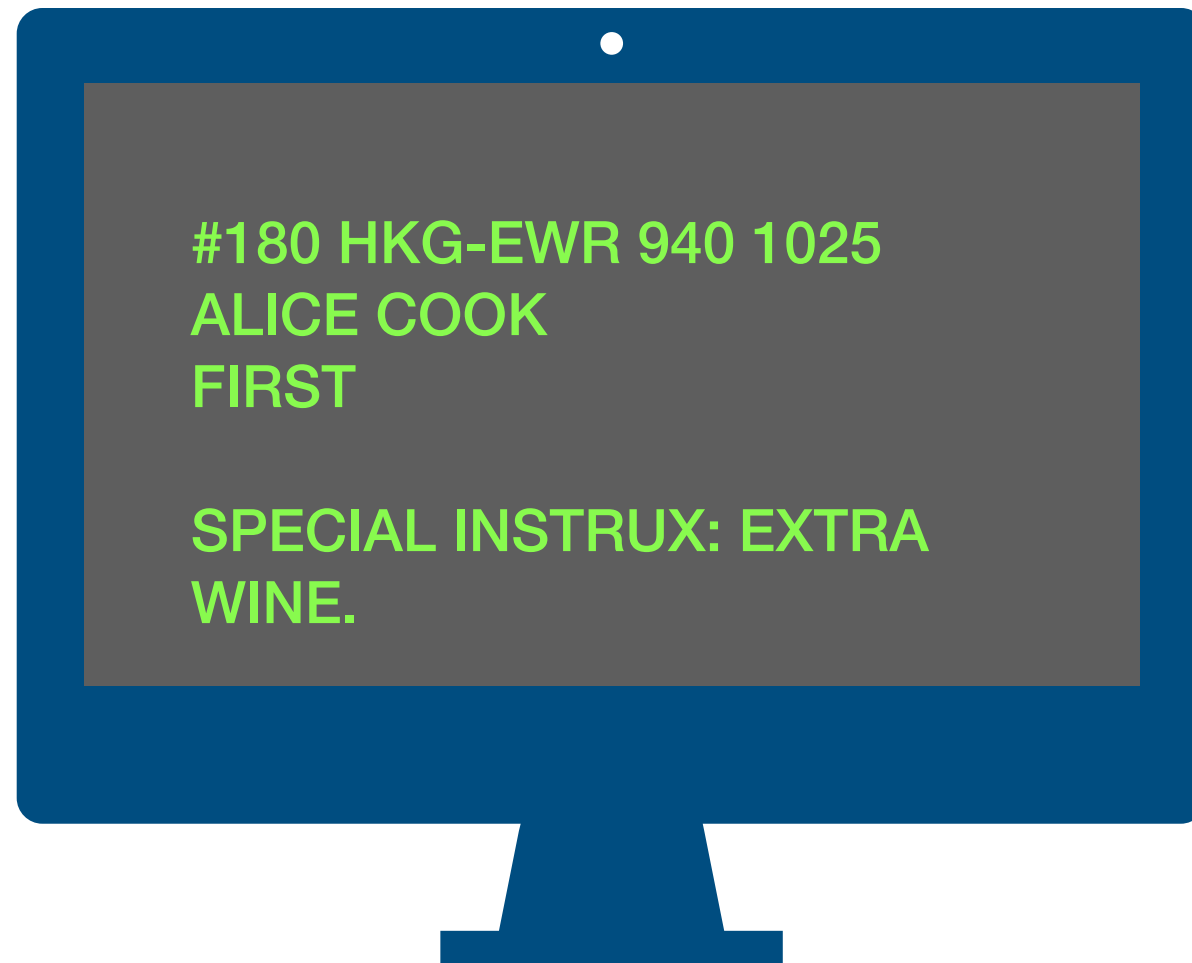


01

2000

► Known Traveler Number and Redress Number

Overflows - Example (Cont.)



Adult (18-64)

Alice

Cook

First

Extra wine.

Date of birth*

01 - January



01

2000

► Known Traveler Number and Redress Number

Overflows - Example (Cont.)

```
void vulnerable() {  
    char name[64];  
    ...  
    gets(name);  
    ...  
}
```

```
void vulnerable() {  
    int first_class = 0;  
    char name[64];  
    ...  
    gets(name);  
    ...  
}
```

```
void vulnerable() {  
    char instrux[128] = "none";  
    int first_class = 0;  
    char name[64];  
    ...  
    gets(name);  
    ...  
}
```

```
void vulnerable() {  
    int authenticated = 0;  
    char name[64];  
    ...  
    gets(name);  
    ...  
}
```

```
void vulnerable() {  
    void (*fp)();  
    char buf[64];  
    ...  
    gets(buf);  
    ...  
}
```

```
void vulnerable() {  
    char command[] = "/bin/cat";  
    char buf[64];  
    ...  
    gets(buf);  
    ...  
    execv(command, ...);  
}
```

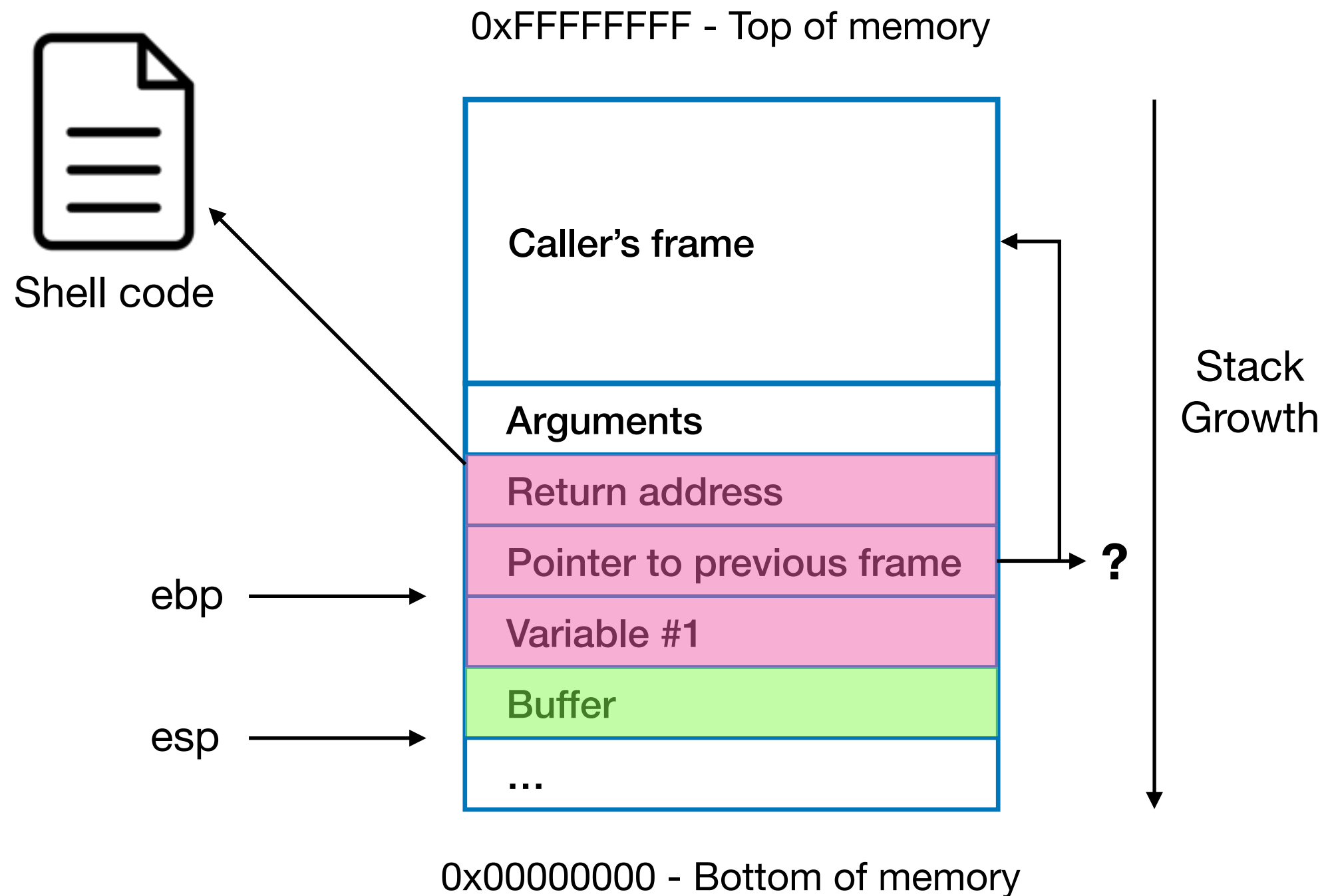
Stack Overflow

- A function's local variables (buffers) are statically allocated on the **stack**
- Data is copied/written without checking boundaries
- Data can overflow a buffer and overwrite other important data on the stack
 - Return address
 - Saved EBP
 - Other?
- Normally it causes a segmentation fault

Stack Overflow (Cont.)

- If correctly crafted, it is possible to
 - Overwrite security sensitive data
 - Overwrite the return address
 - Jump to user-defined code
 - Jump to other code
- The user-defined code **can** be part of the overflowing data
- The code will be executed with the **privilege of the overflowed application**

Stack Overflow (Cont.)



Walking through Overflow

- See separate slides from Vern Paxson

Vulnerable Functions

- `gets()`
 - Data cannot contain newlines or EOFs
- `strcpy()/strcat()`
- `sprintf()/vsprintf()`
- `scanf()/sscanf()/fscanf()`
- Custom input routines

Overflow Opportunities

- Any reference to a value that can be overwritten can represent a security vulnerability
- What can be overwritten?
 - Data & Data pointer
 - Return address
 - Stack frame pointer
 - Exception handler
 - Function pointer
 - Jump destination

Overflow Opportunities (Cont.)

- What causes the overwrite?
 - Copying overflows
 - Array indexing overflows
 - Integer overflows
 - Loop overflows
- Where is overwritten?
 - Stack
 - Heap
 - And more

Safe Functions

- We should use safe version of memory access functions to make sure important data structures cannot be overwritten

```
void safe() {  
    char buf[64];  
    ...  
    fgets(buf, 64, stdin);  
    ...  
}
```

```
void safer() {  
    char buf[64];  
    ...  
    fgets(buf, sizeof(buf), stdin);  
    ...  
}
```

```
void safe(char *data) {  
    char buf[64];  
    ...  
    strncpy(buf, data, sizeof(buf));  
    ...  
}
```

- Many “safe” libc functions are misleading
 - `strncpy()` may leave string unterminated
 - If adjacent buffers are not NULL-terminated, it is possible to copy an excessive amount of data

More Overflows

- Is this function safe?

```
void function(char *data, int len) {  
    char buf[64];  
    if (len > 64)  
        return;  
    memcpy(buf, data, len);  
}
```

```
void *memcpy (void *destination, const void *source, size_t num);
```



Alias of one of the fundamental **unsigned** integer types.

```
#include <stdio.h>  
  
int main(int argc, char *argv[]) {  
    int len = -1;  
    size_t len_ = (size_t)len;  
    printf("len_=0x%x, %u", len_, len_);  
}
```

```
void safe(char *data, size_t len) {  
    char buf[64];  
    if (len > 64)  
        return;  
    memcpy(buf, data, len);  
}
```

More Overflows (Cont.)

- Is this function safe?

```
void function(char *data, size_t len) {  
    char *buf;  
    buf = malloc(len+2);  
    if (buf == NULL)  
        return;  
    memcpy(buf, data, len);  
}
```

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    size_t len = 0xffffffff;  
    size_t len_ = len + 2;  
    printf("len_=0x%x, %u\n", len_, len_);  
}
```

Integer Overflows

- Integer overflows are caused by unexpected results when comparing, casting, and adding/subtracting integers
- Lessons learned
 - A large signed integer can become a large negative value
 - A large unsigned integer can become a small unsigned integer
 - A negative signed integer can become a large unsigned integer
 - Integer of different sizes/types should be assigned with great care
 - Use assertions and similar checks to avoid overflows

Index Overflow

- Index overflows exploit the lack of boundary checks in referencing a value in an array with an index
- They allow for **direct write** to memory locations
- Depending on the type of the array, it is possible to modify only memory values that conform to the type of elements in the array
 - A **char** * array allows for changing **1** byte at a time

Index Overflow - Example

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int array[10];
    int index, value;

    if (argc < 3)
        return 0;

    index = (int) strtol(argv[1], NULL, 10);
    value = (int) strtoul(argv[2], NULL, 16);

    array[index] = value;
    return 0;
}
```

Format String Vulnerabilities

- Are these functions safe?

```
void function(char *name) {  
    printf("Name: %s", name);  
}
```

```
void function(char *buf) {  
    printf(buf);  
}
```

```
void function(char *name) {  
    printf("Name: %s");  
}
```

```
void function(char *name) {  
    printf("Name: %s", name, "%s");  
}
```

```
void function(int score) {  
    printf("Score: %d", score);  
}
```

```
void function(int score) {  
    printf("Score: %d");  
}
```

```
void function(int score) {  
    printf("Score: %d %d %d");  
}
```

```
void function(int score) {  
    printf("Score: %n");  
}
```

More about printf()

- `%n` writes the number of characters printed so far by `printf()` into the address specified by the corresponding argument

Lessons Learned

- Whenever a format string function is used with user input, it is possible to **read** and/or **write** the process memory
 - `printf()`, `fprintf()`, `sprintf()`, ...
 - `vprintf()`, `vfprintf()`, `vsprintf()`, ...
 - `syslog()`, `err()`, `warn()`, `verr()`, `vwarn()`, ...
- Read memory
 - `printf("%08x\n%08x\n%08x\n|s|");`
- Write memory
 - `printf("%08x\n%08x\n%n%08x%n");`
- Safe
 - `printf("%s", user_input);`

Overwriting C++ Vtables

- C++ objects can be dynamically allocated on the heap
- Each object is associated with a pointer (**vfptr**) pointing to the virtual function table (**Vtable**), which is a table of pointers to the definition of the object's (virtual) methods
- A variable overflow can be used to overwrite the **vfptr** so that it points to a fake **Vtable** specified by the attacker
- When a method is invoked the attacker's code is executed

Credits

- Some slides are based on the slides of Giovanni Vigna and Vern Paxson